

Unit 11

Signed Number Systems and Arithmetic

Outline (for Study and Review)

Signed Systems

- What is the difference between Signed Magnitude and 2's complement system
- How do you take the negative of a number in either system

Arithmetic

- How do you perform addition and subtraction on unsigned and 2's complement numbers
- Know (memorize) the tests/rules to determine if overflow has occurred

BINARY REPRESENTATION SYSTEMS REVIEW

Interpreting Binary Strings

- Given a string of 1's and 0's, you need to know the *representation system* being used, before you can understand the value of those 1's and 0's.
- Information (value) = Bits + Context (System)

01000001 = ?

Unsigned
Binary system



65_{10}

BCD System



41_{BCD}

ASCII
system



'A'_{ASCII}

Binary Representation Systems

- Integer Systems

- Unsigned
 - Unsigned (Normal) binary
- Signed
 - Signed Magnitude
 - 2's complement System
 - Excess- N^*
 - 1's complement *

- Floating Point *

- For very large and small (fractional) numbers

- Codes

- Text
 - ASCII / Unicode
- Decimal Codes
 - BCD (Binary Coded Decimal) / (8421 Code)

* = Not fully covered in this class

Signed Magnitude
2's Complement System

SIGNED SYSTEMS

Unsigned and Signed

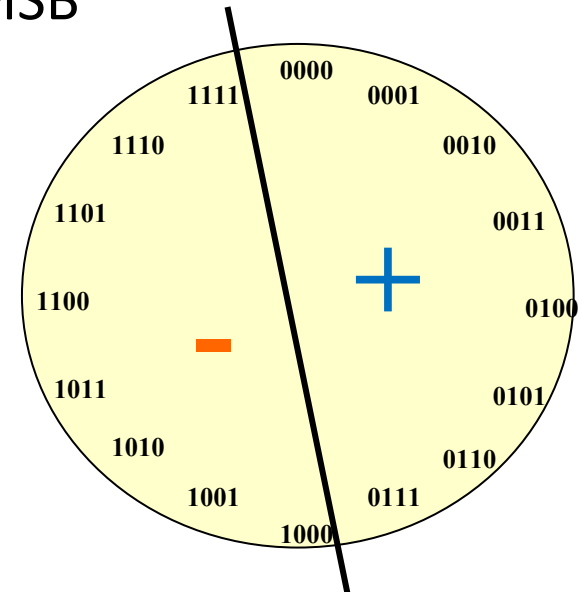
- Normal (unsigned) binary can only represent positive numbers
 - All place values are positive
- To represent BOTH positive and negative numbers we must use the available binary codes differently, some for the positive values and others for the negative values
 - We call these *signed* representations

Signed Number Representation

- 2 Primary Systems
 - Signed Magnitude
 - *Two's Complement* (most widely used for integer representation)

Signed numbers

- All systems used to represent negative numbers split the possible binary combinations in half (half for positive numbers / half for negative numbers)
- In both **signed magnitude** and **2's complement**, positive and negative numbers are separated using the MSB
 - **MSB=1 means negative**
 - **MSB=0 means positive**



Summarizing 3 Key Systems

- Below is a side-by-side comparison of the 3 key systems to know for unsigned and signed integer representation [**Differ in the meaning of the MSB**]
 - Unsigned** (positive only => Positive powers of 2)
 - Sign and Magnitude** (aka Signed Magnitude): MSB is +/- sign (**0=pos, 1=neg**)
 - 2's complement system** (Use powers of 2 but MSB has negative weight: -2^{n-1})



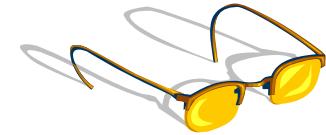
Bit 3	Bit 2	Bit 1	Bit 0	
1	1	0	1	= 13
8	4	2	1	

4-bit Unsigned



Bit 3	Bit 2	Bit 1	Bit 0	
1	1	0	1	= -5
+/-	4	2	1	

4-bit Signed Magnitude



Bit 3	Bit 2	Bit 1	Bit 0	
1	1	0	1	= -3
-8	4	2	1	

4-bit 2's complement System

4-bit Number Comparison

- Below is a side-by-side comparison of the 3 key systems to know for unsigned and signed integer representation [**Differ in the meaning of the MSB**]

4-bit Unsigned

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= 15
8	4	2	1	
1	0	1	0	= 10
8	4	2	1	
0	0	1	1	= 3
8	4	2	1	
0	1	0	1	= 5
8	4	2	1	

4-bit Signed Magnitude

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -7
+/-	4	2	1	
1	0	1	0	= -2
+/-	4	2	1	
0	0	1	1	= +3
+/-	4	2	1	
0	1	0	1	= +5
+/-	4	2	1	

4-bit 2's complement System

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -1
-8	4	2	1	
1	0	1	0	= -6
-8	4	2	1	
0	0	1	1	= +3
-8	4	2	1	
0	1	0	1	= +5
-8	4	2	1	

8-bit Number Comparison

- Interpret these 8-bit numbers in **unsigned**, **signed magnitude**, and **2's complement** system

4-bit Unsigned

147

4-bit Signed
Magnitude

-19

8-bit 2's complement
System

-109

1	0	0	1	0	0	1	1
128	64	32	16	8	4	2	1
+/-							

-128

4-bit Unsigned

25

4-bit Signed
Magnitude

25

8-bit 2's complement
System

25

0	0	0	1	1	0	0	1
128	64	32	16	8	4	2	1
+/-							
-128							

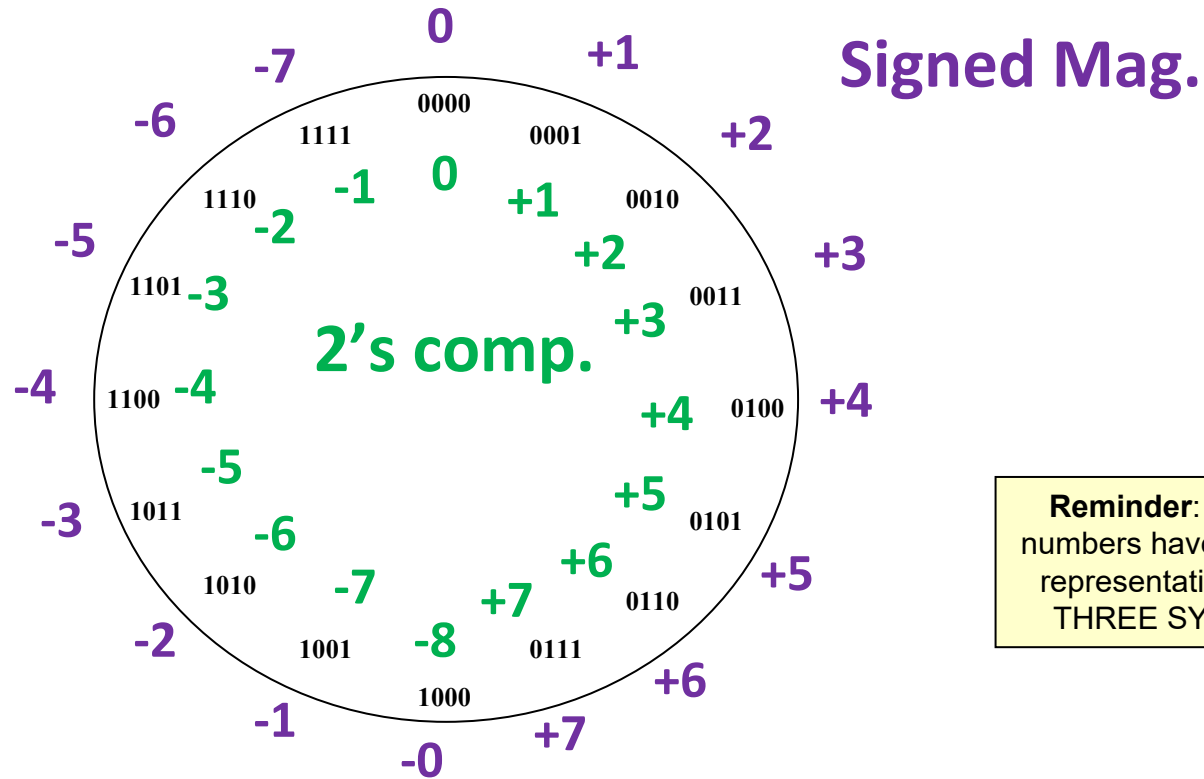
Important: Positive numbers have the same representation in ALL THREE SYSTEMS.

System Ranges

- Compare the number ranges of the 3 systems for n-bits.

						Unsigned	4-bit Signed Magnitude	2's complement System
Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0			
						4-bit Range: 0 to 15	4-bit Range: -7 to +7	4-bit Range: -8 to +7
						6-bit Range: 0 to 63	6-bit Range: -31 to +31	6-bit Range: -32 to +31
						<u>n-bit Range:</u> [0 to $2^n - 1$]	<u>n-bit Range:</u> [$-(2^{n-1} - 1)$ to $+2^{n-1} - 1$]	<u>n-bit Range:</u> [-2^{n-1} to $+2^{n-1} - 1$]

Comparison of Signed Systems



Reminder: Positive numbers have the same representation in ALL THREE SYSTEMS.

Disadvantages of Signed Magnitude

1. Wastes a combination to represent -0

$$0000 = 1000 = 0_{10}$$

2. Addition and subtraction algorithms for signed magnitude are different than unsigned binary (we'd like them to be the same to use the same hardware).

$$\begin{array}{r}
 4 \\
 - 6 \\
 \hline
 \end{array}
 \quad
 \begin{array}{c}
 \text{Swap \& make} \\
 \text{result} \\
 \text{negative}
 \end{array}
 \quad
 \begin{array}{r}
 6 \\
 - 4 \\
 \hline
 \end{array}$$

4-bit Unsigned

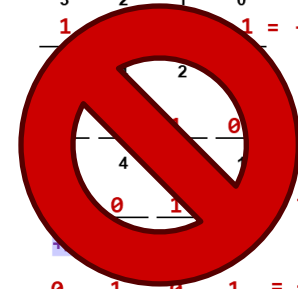
Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= 15
8	4	2	1	
1	0	1	0	= 10
8	4	2	1	
0	0	1	1	= 3
8	4	2	1	
0	1	0	1	= 5
8	4	2	1	

4-bit Signed Magnitude

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -7
1	0	1	0	= -2
0	1	0	1	= +3
0	1	0	1	= +5
+/-	4	2	1	

2's complement System

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -1
-8	4	2	1	
1	0	1	0	= -6
-8	4	2	1	
0	0	1	1	= +3
-8	4	2	1	
0	1	0	1	= +5
-8	4	2	1	



The Winner: 2's Complement System

We have a winner!
2's complement is the preferred "SIGNED"
representation system.

- All computer systems use the 2's complement system to represent signed integers!
- So, from now on, if we say an integer is "signed", we are saying it uses the 2's complement system unless otherwise specified
 - We will **NOT** use "signed magnitude" unless **explicitly** indicated

Unsigned and Signed Variables

- In C, unsigned variables use **unsigned binary** (normal power-of-2 place values) to represent numbers

`unsigned char x = 147;`

1	0	0	1	0	0	1	1	= +147
128	64	32	16	8	4	2	1	

- In C, signed variables use the **2's complement system** (Neg. MSB weight) to represent numbers

`char x = -109;`

1	0	0	1	0	0	1	1	= -109
-128	64	32	16	8	4	2	1	

Key Numbers in 2's Complement

- What are the 1 and 0 patterns for these numbers in 2's complement?

	Bit n-1	Bit n-2		Bit 2	Bit 1	Bit 0	
Weights	-2^{n-1}	2^{n-2}		4	2	1	
Max magnitude positive :	<u>0</u>	<u>1</u>	...	<u>1</u>	<u>1</u>	<u>1</u>	$2^{n-1} - 1$
Max magnitude negative :	<u>1</u>	<u>0</u>	...	<u>0</u>	<u>0</u>	<u>0</u>	-2^{n-1}
What number is this?	<u>1</u>	<u>1</u>	...	<u>1</u>	<u>1</u>	<u>1</u>	-1

Largest positive is
always: 0 followed
by all 1s

Largest negative is
always: 1 followed
by all 0s

All 1s (or hex Fs) is
always -1

Sign and Zero Extension

- **Task:** Keep the numeric value the same but use more bits:
 - **Unsigned:** Always extend with **0s**. (aka **ZERO EXTENSION**)
 - **2's complement:** Replicate the **MSB / sign bit** (aka **SIGN EXTENSION**)

Unsigned

Bit 3	Bit 2	Bit 1	Bit 0	
0	0	1	1	= +3

8 4 2 1

0	0	0	0	0	0	1	1	= +3
---	---	---	---	---	---	---	---	------

128 64 32 16 8 4 2 1

1	1	0	1	= 13
---	---	---	---	------

8 4 2 1

0	0	0	0	1	1	0	1	= 13
---	---	---	---	---	---	---	---	------

128 64 32 16 8 4 2 1

2's complement System

Bit 3	Bit 2	Bit 1	Bit 0	
0	0	1	1	= +3

-8 4 2 1

0	0	0	0	0	0	1	1	= +3
---	---	---	---	---	---	---	---	------

-128 64 32 16 8 4 2 1

1	1	0	1	= -3
---	---	---	---	------

-8 4 2 1

1	1	1	1	1	1	0	1	= -3
---	---	---	---	---	---	---	---	------

-128 64 32 16 8 4 2 1

Sign and Zero Truncation

- **Task:** Without changing the numeric value, remove as many leading bits as possible
 - **Unsigned:** Remove any amount of leading **0s**. (aka **ZERO TRUNCATION**)
 - **2's complement:** Remove **COPIES** of the **MSB / sign bit** (aka **SIGN TRUNCATION**)

				Bit 3	Bit 2	Bit 1	Bit 0	
								
0	0	0	1	0	0	1	1	= +19
128	64	32	16	8	4	2	1	
			1	0	0	1	1	= +19
		16	8	4	2	1		
								
0	0	0	0	0	1	0	1	= 5
128	64	32	16	8	4	2	1	
					1	0	1	= 5
				4	2	1		

Unsigned

				Bit 3	Bit 2	Bit 1	Bit 0		
									
0	0	0	0	1	1	0	0	= +12	
-128	64	32	16	8	4	2	1		
			0	1	1	0	0	= +12	
			-16	8	4	2	1		
									
1	1	1	1	1	1	0	1	= -3	
-128	64	32	16	8	4	2	1		
						1	0	1	= -3
					1	0	1	= -3	
					-4	2	1		

2's complement System

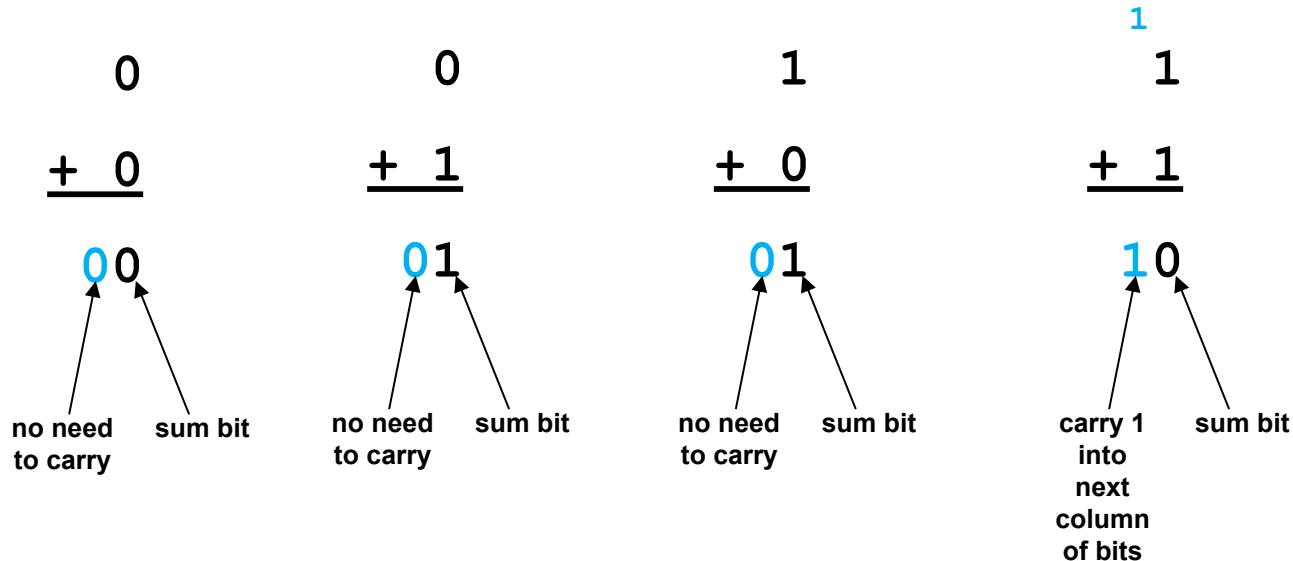
ARITHMETIC

Binary Arithmetic

- Can perform all arithmetic operations (+, -, *, ÷) on binary numbers
- Can use same methods as in decimal
 - Still use carries and borrows, etc.
 - Only now we carry when sum is 2 or more rather than 10 or more (decimal)
 - We borrow 2's not 10's from other columns
- Easiest method is to add bits in your head in decimal ($1+1 = 2$) then convert the answer to binary ($2_{10} = 10_2$)

Binary Addition

- In **decimal** addition we carry when the sum is **10** or more
- In **binary** addition we carry when the sum is **2** or more
- Add bits in binary to produce a sum bit and a carry bit



Binary Addition & Subtraction

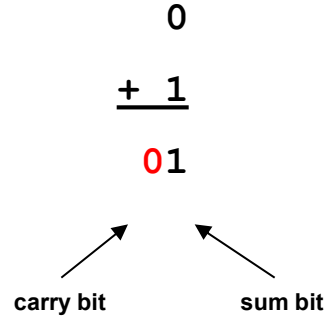
$$\begin{array}{r}
 \begin{array}{cccc}
 1 & 1 & 1 & \\
 0 & 1 & 1 & 1
 \end{array} (7) \\
 + \begin{array}{cccc}
 0 & 0 & 1 & 1
 \end{array} (3) \\
 \hline
 \begin{array}{cccc}
 1 & 0 & 1 & 0
 \end{array} (10) \\
 \begin{array}{cccc}
 8 & 4 & 2 & 1
 \end{array}
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{cccc}
 0 & & 0 & \\
 \cancel{1} & 10 & \cancel{1} & 10
 \end{array} (10) \\
 - \begin{array}{cccc}
 0 & 1 & 0 & 1
 \end{array} (5) \\
 \hline
 \begin{array}{cccc}
 0 & 1 & 0 & 1
 \end{array} (5) \\
 \begin{array}{cccc}
 8 & 4 & 2 & 1
 \end{array}
 \end{array}$$

Binary Addition

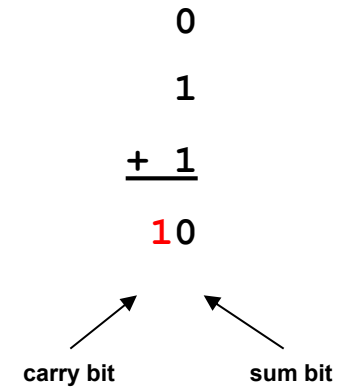
1

$$\begin{array}{r}
 0 \\
 0110 \text{ (6)} \\
 + 0111 \text{ (7)} \\
 \hline
 1101 \text{ (13)}
 \end{array}$$



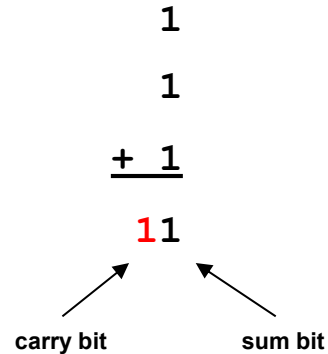
2

$$\begin{array}{r}
 10 \\
 0110 \text{ (6)} \\
 + 0111 \text{ (7)} \\
 \hline
 1101 \text{ (13)}
 \end{array}$$



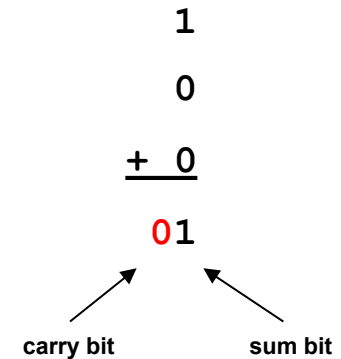
3

$$\begin{array}{r}
 110 \\
 0110 \text{ (6)} \\
 + 0111 \text{ (7)} \\
 \hline
 1101 \text{ (13)}
 \end{array}$$



4

$$\begin{array}{r}
 110 \\
 0110 \text{ (6)} \\
 + 0111 \text{ (7)} \\
 \hline
 1101 \text{ (13)}
 \end{array}$$



Hexadecimal Arithmetic

- Same style of operations
 - Carry when sum is 16 or more, etc.

$$\begin{array}{r} 1 \ 1 \\ 4 \ D_{16} \end{array}$$

$$+ \ B \ 5_{16}$$

$$1 \ 0 \ 2_{16}$$

$$13+5 = 18_{10} = \underline{1 \ 2}_{16}$$

16 1

$$1+4+11 = 16_{10} = \underline{1 \ 0}_{16}$$

16 1

"Taking the 2's complement"

TAKING THE NEGATIVE

Taking the Negative

- Given, x , what bit operations will yield $-x$.

Signed Magnitude

Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
1	1	0	0	1	1	= -19
+/-	16	8	4	2	1	

Given x , to find $-x$ just flip the MSB

Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
0	1	0	0	1	1	= +19
+/-	16	8	4	2	1	

2's Complement System

Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
1	0	1	1	0	1	= -19
-32	16	8	4	2	1	

0	0	1	0	0	1	0
+						1

0	0	1	1	0	1
-32	4	8	4	2	1

Operation for taking the negative

1. Flip all bits ($\sim x$)

2a. Add 1

2b. Drop final carry

This is called "Taking the 2's complement"

+19 =

Taking the 2's Complement

- To take the negative of a number represented in the 2's complement system...
- ...**take the 2's complement** (an operation to take the negative)
 - Defined as $\sim x + 1$ (i.e. **bit flip of x plus 1**)
 - **Drop any final carry** (if you start with n-bits, end with n-bits)

-32 16 8 4 2 1
010011

Original number = +19



101100

$\sim x + 1$

+ 1

101101

Resulting number = -19

Important: Taking the 2's complement is equivalent to taking the negative (negating)

2's Comp. System vs. Taking the 2's comp.

$$1110 = -2$$

$$?? = +2$$

2's comp. system

$$0001 = \sim x$$

$$+ \quad 1 = +1$$

$$0010 = +2$$

"Taking the
2's comp."
(find $-x$)

2's comp. system

$$0101 = +5$$

$$?? = -5$$

2's comp. system

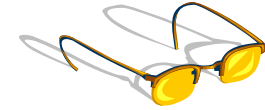
$$1010 = \sim x$$

$$+ \quad 1 = +1$$

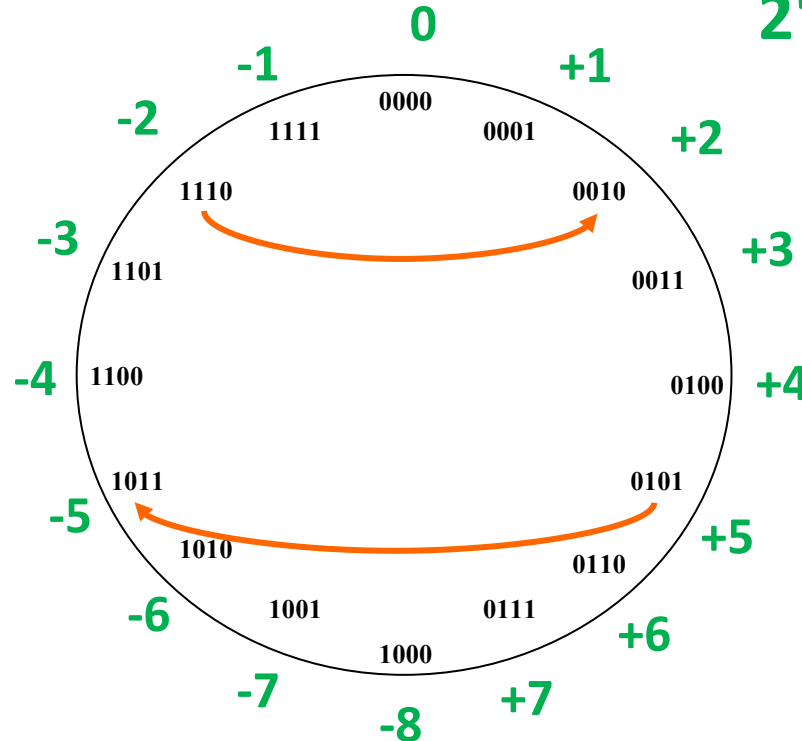
$$1011 = -5$$

"Taking the
2's comp."
(find $-x$)

2's comp. system



2's comp. system



The **2's complement system** is a way of **interpreting bits** as signed numbers (MSB has negative weight).

"**Taking the 2's complement**" is an **operation** used to **find the negative** representation of a number, x .

Taking the 2's Complement

1

-32 16 8 4 2 1	
101010	Original number = -22
010101	Take the 2's complement yields the negative of a number
+ 1	
010110	
	Resulting number = +22
101001	Taking the 2's complement again yields the original number (the operation is symmetric)
+ 1	
101010	
	Back to original = -22

2

0000	Original # = 0	
1111	Take the 2's complement	
+ 1		
0000		
	2's comp. of 0 is 0	
3	1000	Original # = -8
0111	Take the 2's complement	
+ 1		
1000		
	Negative of -8 is -8 (i.e. no positive equivalent, but this is not a huge problem)	

Subtraction the easy way (via the 2's complement)

ADDITION AND SUBTRACTION

2's Complement Addition/Subtraction

- Addition
 - Sign of the numbers do not matter
 - Add column by column
 - Drop any final carry-out
- Subtraction
 - Any subtraction ($A-B$) can be converted to addition ($A + -B$) by taking the 2's complement of B
 - ($A-B$) becomes ($A + \sim B + 1$)
 - Drop any carry-out
- The sign of the result is produced by performing the above process and need not be considered separately

2's Complement Addition

- No matter the sign of the operands just add as normal
- Drop any extra carry out

$$\begin{array}{r}
 0000 \\
 0011 \text{ (3)} \\
 + 0010 \text{ (2)} \\
 \hline
 0101 \text{ (5)}
 \end{array}$$

$$\begin{array}{r}
 0000 \\
 1101 \text{ (-3)} \\
 + 0010 \text{ (2)} \\
 \hline
 1111 \text{ (-1)}
 \end{array}$$

Drop final carry-out →

$$\begin{array}{r}
 \cancel{1}110 \\
 0011 \text{ (3)} \\
 + 1110 \text{ (-2)} \\
 \hline
 0001 \text{ (1)}
 \end{array}$$

$$\begin{array}{r}
 \cancel{1}100 \\
 1101 \text{ (-3)} \\
 + 1110 \text{ (-2)} \\
 \hline
 1011 \text{ (-5)}
 \end{array}$$

Unsigned and Signed Addition

- Addition process is the same for both unsigned and signed numbers
 - Add columns right to left
- Examples:

	<u>If unsigned</u>	<u>If signed</u>
$ \begin{array}{r} 11 \\ 1001 \\ + 0011 \\ \hline 1100 \end{array} $	$ \begin{array}{r} (9) \\ (3) \\ (12) \end{array} $	$ \begin{array}{r} (-7) \\ (3) \\ (-4) \end{array} $

2's Complement Subtraction

- Take the 2's complement of the subtrahend and add to the original minuend
- Drop any extra carry out

$$\begin{array}{r} 0011 \text{ (+3)} \\ - 0010 \text{ (+2)} \\ \hline \end{array}$$

$$\begin{array}{r} 1101 \text{ (-3)} \\ - 1110 \text{ (-2)} \\ \hline \end{array}$$

Drop final carry-out →

$$\begin{array}{r} \cancel{1}111 \\ 0011 \quad A \\ 1101 \quad \sim B \\ + \quad 1 \quad \text{Add 1} \\ \hline 0001 \end{array}$$

$$\begin{array}{r} 1 \\ 1101 \quad A \\ 0001 \quad \sim B \\ + \quad 1 \quad \text{Add 1} \\ \hline 1111 \end{array}$$

Unsigned and Signed Subtraction

- Subtraction process is the same for both unsigned and signed numbers
 - Convert $A - B$ to $A + \text{Comp. of } B$
 - Drop any final carry out
- Examples:

	<u>If unsigned</u>	<u>If signed</u>			<u>If unsigned</u>	<u>If signed</u>
1100	(12)	(-4)	➔	11 1 1100	A	
- 0010	(2)	(2)		1101	~B	
<u> </u>				+ 1	Add 1	
				<u> </u>		
				1010	(10)	(-6)

Important Note

- Almost all computers use 2's complement because...
- The same addition and subtraction algorithm can be used on unsigned and 2's complement (signed) numbers
- Thus, we only need one adder circuit (hardware component) to perform operations on both unsigned and signed numbers

OVERFLOW

Overflow

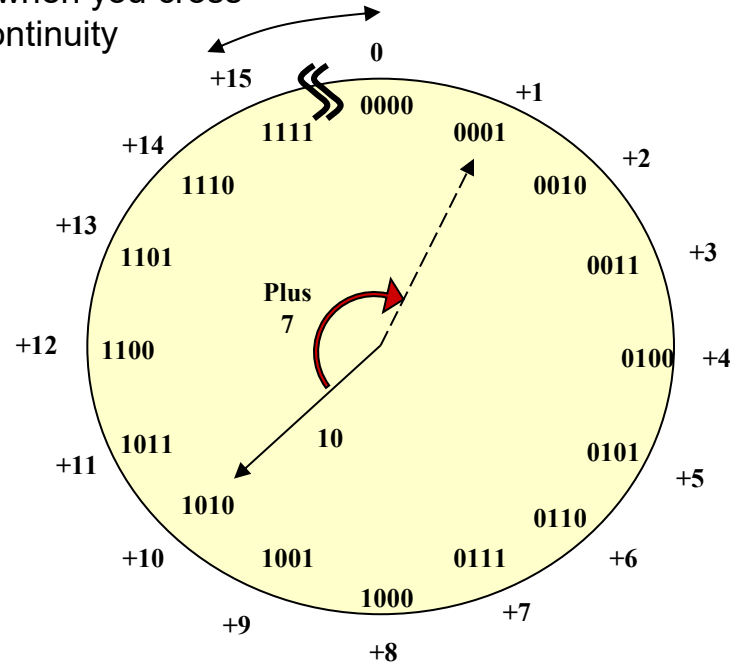
- Overflow occurs when the result of an arithmetic operation is too large to be represented with the given number of bits
- Conditions and tests to determine overflow depend on type/system of numbers (signed or unsigned) in the operation

Unsigned Overflow

Overflow occurs when you cross this discontinuity

$$10 + 7 = 17$$

With 4-bit *unsigned* numbers we can only represent 0 – 15. Thus, we say overflow has occurred.

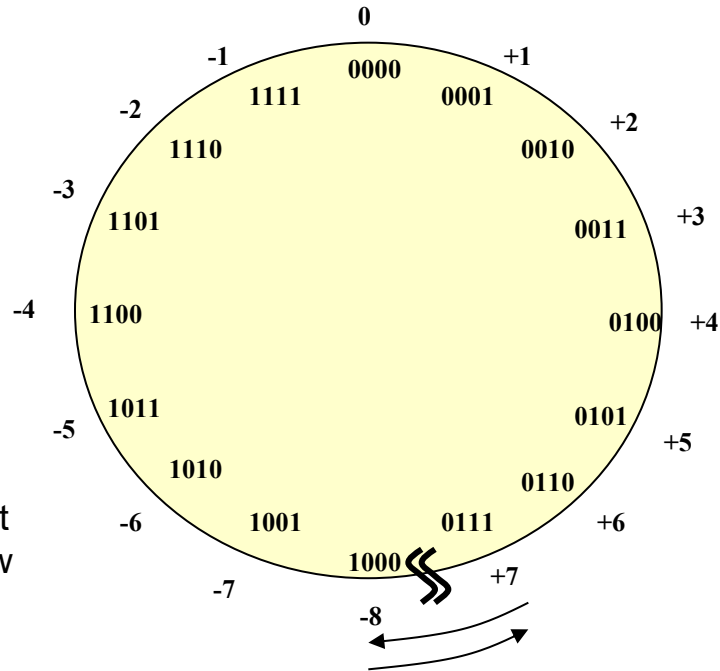


2's Complement Overflow

$$5 + 7 = +12$$

$$-6 + -4 = -10$$

With 4-bit 2's complement numbers we can only represent -8 to +7. Thus, we say overflow has occurred.



Overflow occurs when you cross this discontinuity

Overflow in Addition

- Overflow occurs when the result of the addition cannot be represented with the given number of bits.
- Tests for overflow:
 - Unsigned: if Cout = 1
 - Signed: if $p + p = n$ or $n + n = p$

	<u>If unsigned</u>	<u>If signed</u>
1 1		
1101	(13)	(-3)
+ 0100	(4)	(4)
0001	(1)	(+1)

Overflow: y
Cout = 1

Overflow: n
 $n + p$

	<u>If unsigned</u>	<u>If signed</u>
0 1		
0110	(6)	(6)
+ 0101	(5)	(5)
1011	(11)	(-5)

Overflow: n
Cout = 0

Overflow: y
 $p + p = n$

Overflow in Subtraction

- Overflow occurs when the result of the subtraction cannot be represented with the given number of bits.
- Tests for overflow:
 - Unsigned: if Cout = 0
 - Signed: after converting to addition, if $p + p = n$ or $n + n = p$

	<u>If unsigned</u>	<u>If signed</u>
0111	(7)	(7)
- 1000	(8)	(-8)
<u> </u>	(-1)	(15)

Desired
Results



0111	
0111	A
0111	~B
+ 1	Add 1
<u> </u>	
1111	(15) (-1)

If unsigned
Overflow: y
Cout = 0

If signed
Overflow: y
 $p + p = n$

FLOATING POINT

Floating Point

- Used to represent very small numbers (fractions) and very large numbers
 - Avogadro's Number: $+6.0247 * 10^{23}$
 - Planck's Constant: $+6.6254 * 10^{-27}$
 - Note: 32 or 64-bit integers can't represent this range
- Floating Point representation is used in HLL's like C by declaring variables as **float** or **double**

Fixed Point

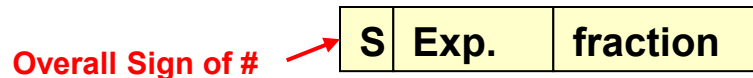
- Unsigned and 2's complement fall under a category of representations called "Fixed Point"
- The radix point is assumed to be in a fixed location for all numbers [Note: we could represent fractions by implicitly assuming the binary point is at the left...A variable just stores bits...you can assume the binary point is anywhere you like]
 - Integers: **10011101.** (binary point to right of LSB)
 - For 32-bits, unsigned range is 0 to ~4 billion
 - Fractions: **.10011101** (binary point to left of MSB)
 - Range [0 to 1)
- **Main point:** By fixing the radix point, we limit the range of numbers that can be represented
 - Floating point allows the radix point to be in a different location for each value

Bit storage

Fixed point Rep.

Floating Point Representation

- Similar to scientific notation used with decimal numbers
 - $\pm D.DDD * 10^{\pm \text{exp}}$
- Floating Point representation uses the following form
 - $\pm b.bbbb * 2^{\pm \text{exp}}$
 - 3 Fields: sign, exponent, fraction (also called mantissa or significand)



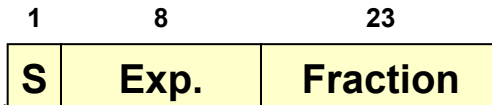
Normalized FP Numbers

- Decimal Example
 - $+0.754 \cdot 10^{15}$ is not correct scientific notation
 - Must have exactly one significant digit before decimal point: $+7.54 \cdot 10^{14}$
- In binary the only significant digit is '1'
- Thus normalized FP format is:
$$\pm 1.\text{bbbbbb} \cdot 2^{\pm \text{exp}}$$
- FP numbers will **always be normalized** before being stored in memory or a reg.
 - The **1.** is actually not stored but assumed since we always will store normalized numbers
 - If HW calculates a result of $0.001101 \cdot 2^5$ it must normalize to $1.101000 \cdot 2^2$ before storing

IEEE Floating Point Formats

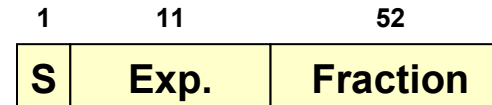
- Single Precision
(32-bit format)

- 1 Sign bit (0=pos/1=neg)
- 8 Exponent bits
 - Excess-127 representation
 - More on next slides
- 23 fraction (significand or mantissa) bits
- Equiv. Decimal Range:
 - 7 digits x $10^{\pm 38}$



- Double Precision
(64-bit format)

- 1 Sign bit (0=pos/1=neg)
- 11 Exponent bits
 - Excess-1023 representation
 - More on next slides
- 52 fraction (significand or mantissa) bits
- Equiv. Decimal Range:
 - 16 digits x $10^{\pm 308}$



Floating Point vs. Fixed Point

- Single Precision (32-bits) Equivalent Decimal Range:
 - 7 significant decimal digits * $10^{\pm 38}$
 - Compare that to 32-bit signed integer where we can represent ± 2 billion. How does a 32-bit float allow us to represent such a greater range?
 - FP allows for **range** but sacrifices **precision** (can't represent all numbers in its range)
- Double Precision (64-bits) Equivalent Decimal Range:
 - 16 significant decimal digits * $10^{\pm 308}$

Exponent Representation

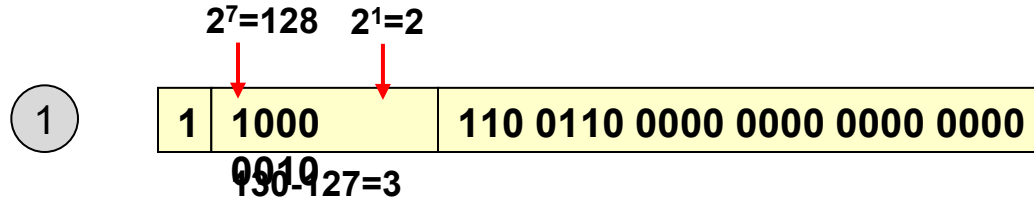
- Exponent needs its own sign (+/-)
- Rather than using 2's comp. system we use Excess-N representation
 - Single-Precision uses Excess-127
 - Double-Precision uses Excess-1023
 - This representation allows FP numbers to be easily compared
- Let E' = stored exponent code and E = true exponent value
- For single-precision: $E' = E + 127$
 - $2^1 \Rightarrow E = 1, E' = 128_{10} = 10000000_2$
- For double-precision: $E' = E + 1023$
 - $2^{-2} \Rightarrow E = -2, E' = 1021_{10} = 01111111101_2$

2's comp.	E' (stored Exp.)	Excess-127
-1	1111 1111	+128
-2	1111 1110	+127
-128	1000 0000	1
+127	0111 1111	0
+126	0111 1110	-1
+1	0000 0001	-126
0	0000 0000	-127

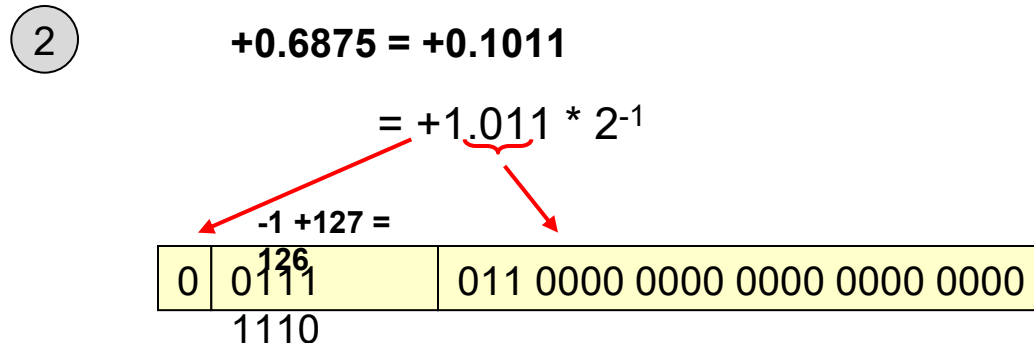
Comparison of
2's comp. & Excess-N

Q: Why don't we use Excess-N more to represent negative #'s

Single-Precision Examples



$$\begin{aligned}
 & -1.1100110 * 2^3 \\
 & = -1110.011 * 2^0 \\
 & = -14.375
 \end{aligned}$$



BACKUP

4-bit Number Comparison

- Below is a side-by-side comparison of the 3 key systems to know for unsigned and signed integer representation [**Differ in the meaning of the MSB**]

4-bit Unsigned

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= 15
8	4	2	1	
1	0	1	0	= 10
8	4	2	1	
0	0	1	1	= 3
8	4	2	1	
0	1	0	1	= 5
8	4	2	1	

4-bit Signed Magnitude

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -7
+/-	4	2	1	
1	0	1	0	= -2
+/-	4	2	1	
0	0	1	1	= +3
+/-	4	2	1	
0	1	0	1	= +5
+/-	4	2	1	

2's complement System

Bit 3	Bit 2	Bit 1	Bit 0	
1	1	1	1	= -1
-8	4	2	1	
1	0	1	0	= -6
-8	4	2	1	
0	0	1	1	= +3
-8	4	2	1	
0	1	0	1	= +5
-8	4	2	1	

Signed Magnitude System

- Use binary place values but now MSB represents the sign (1 if negative, 0 if positive)

4-bit
Unsigned

Bit 3	Bit 2	Bit 1	Bit 0
8	4	2	1



0 to 15

4-bit Signed
Magnitude

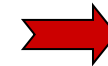
Bit 3	Bit 2	Bit 1	Bit 0
+/-	4	2	1



-7 to +7

8-bit Signed
Magnitude

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
+/-	64	32	16	8	4	2	1



-127 to +127

Signed Magnitude Examples

4-bit Signed Magnitude

$$\begin{array}{c|c|c|c} 1 & 1 & 0 & 1 \\ \hline \end{array} = -5$$

$$\begin{array}{c|c|c|c} +/- & 4 & 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{c|c|c|c} 0 & 0 & 1 & 1 \\ \hline \end{array} = +3$$

$$\begin{array}{c|c|c|c} +/- & 4 & 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{c|c|c|c} 1 & 1 & 1 & 1 \\ \hline \end{array} = -7$$

$$\begin{array}{c|c|c|c} +/- & 4 & 2 & 1 \\ \hline \end{array}$$

Notice that +3 in signed magnitude is the same as in the unsigned system

8-bit Signed Magnitude

$$\begin{array}{c|c|c|c|c|c|c|c} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ \hline \end{array} = -19$$

$$\begin{array}{c|c|c|c|c|c|c|c} +/- & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{c|c|c|c|c|c|c|c} 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ \hline \end{array} = +25$$

$$\begin{array}{c|c|c|c|c|c|c|c} +/- & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ \hline \end{array}$$

Important: Positive numbers have the same representation in signed magnitude as in normal unsigned binary

Signed Magnitude Range

- Given n bits...
 - MSB is sign
 - Other $n-1$ bits = normal unsigned place values
 - Range with $n-1$ unsigned bits = $[0 \text{ to } 2^{n-1}-1]$

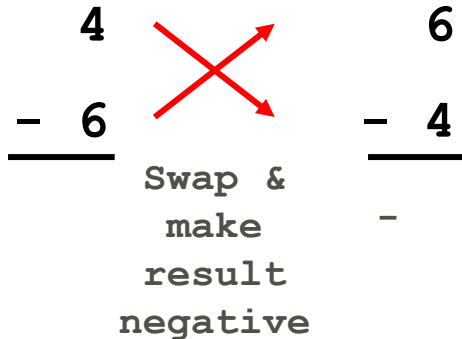
Range with n -bits of Signed Magnitude
 $[-(2^{n-1}-1) \text{ to } +(2^{n-1}-1)]$

Disadvantages of Signed Magnitude

1. Wastes a combination to represent -0

$$0000 = 1000 = 0_{10}$$

2. Addition and subtraction algorithms for signed magnitude are different than unsigned binary (we'd like them to be the same to use the same hardware).



$$\begin{array}{r}
 4 \\
 - 6 \\
 \hline
 \end{array}
 \quad
 \begin{array}{c}
 \text{Swap \&} \\
 \text{make} \\
 \text{result} \\
 \text{negative}
 \end{array}
 \quad
 \begin{array}{r}
 6 \\
 - 4 \\
 \hline
 -
 \end{array}$$

2's Complement System

- Normal binary place values except MSB has negative weight
 - MSB of 1 = -2^{n-1}

4-bit
Unsigned

Bit 3	Bit 2	Bit 1	Bit 0
8	4	2	1



0 to 15

4-bit
2's complement

Bit 3	Bit 2	Bit 1	Bit 0
-8	4	2	1



-8 to +7

8-bit
2's complement

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
-128	64	32	16	8	4	2	1



-128 to +127

2's Complement Examples

4-bit
2's complement

$$\begin{array}{r} \underline{1} \quad \underline{0} \quad \underline{1} \quad \underline{1} \\ -8 \quad 4 \quad 2 \quad 1 \end{array} = -5$$

$$\begin{array}{r} \underline{0} \quad \underline{0} \quad \underline{1} \quad \underline{1} \\ -8 \quad 4 \quad 2 \quad 1 \end{array} = +3$$

$$\begin{array}{r} \underline{1} \quad \underline{1} \quad \underline{1} \quad \underline{1} \\ -8 \quad 4 \quad 2 \quad 1 \end{array} = -1$$

Notice that +3 in 2's comp. is the same as in the unsigned system

8-bit
2's complement

$$\begin{array}{r} \underline{1} \quad \underline{0} \quad \underline{0} \quad \underline{0} \quad \underline{0} \quad \underline{0} \quad \underline{0} \quad \underline{1} \\ -128 \quad 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \end{array} = -127$$

$$\begin{array}{r} \underline{0} \quad \underline{0} \quad \underline{0} \quad \underline{1} \quad \underline{1} \quad \underline{0} \quad \underline{0} \quad \underline{1} \\ -128 \quad 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \end{array} = +25$$

Important: Positive numbers have the same representation in 2's complement as in normal unsigned binary

2's Complement Range

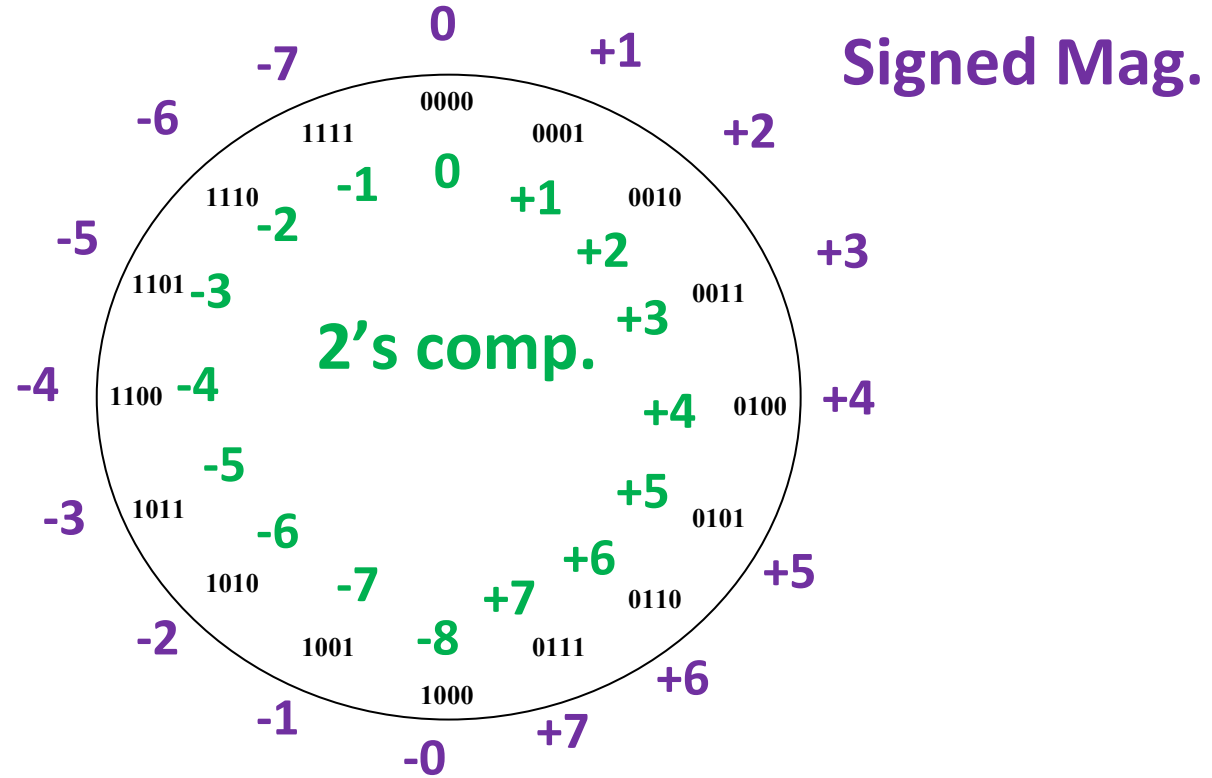
- Given n bits...
 - Max positive value = $011\dots11$
 - Includes all $n-1$ positive place values
 - Max negative value = $100\dots00$
 - Includes only the negative MSB place value

Range with n-bits of 2's complement

$$[-2^{n-1} \text{ to } +2^{n-1}-1]$$

- Side note – What decimal value is $111\dots11$?
 - -1_{10}

Comparison of Systems



Unsigned and Signed Variables

- In C, unsigned variables use **unsigned binary** (normal power-of-2 place values) to represent numbers

`unsigned char x = 147;`

1	0	0	1	0	0	1	1	= +147
128	64	32	16	8	4	2	1	

- In C, signed variables use the **2's complement system** (Neg. MSB weight) to represent numbers

`char x = -109;`

1	0	0	1	0	0	1	1	= -109
-128	64	32	16	8	4	2	1	

IMPORTANT NOTE

- All computer systems use the 2's complement system to represent signed integers!
- So from now on, if we say an integer is signed, we are actually saying it uses the 2's complement system unless otherwise specified
 - We will not use "signed magnitude" unless explicitly indicated

Zero and Sign Extension

- Extension is the process of increasing the number of bits used to represent a number without changing its value
 - Unsigned = Zero Extension (Always add leading 0's):

$$111011 = 00111011$$

↑ Increase a 6-bit number to 8-bit number by zero extending

- 2's complement = Sign Extension (Replicate sign bit):

$$\text{pos} \quad 011010 = 0011010$$

$$\text{neg.} \quad 110011 = 11110011$$

Sign bit is just repeated as many times as necessary

Zero and Sign Truncation

- Truncation is the process of decreasing the number of bits used to represent a number without changing its value
 - Unsigned = Zero Truncation (Remove leading 0's):

$$\cancel{00}111011 = 111011$$

Decrease an 8-bit number to 6-bit number by truncating 0's. Can't remove a '1' because value is changed

- 2's complement = Sign Truncation (Remove copies of sign bit):

$$\text{pos } \cancel{00}111010 = 011010$$

.

$$\text{neg. } \cancel{11}110011 = 10011$$

Any copies of the MSB can be removed without changing the numbers value. Be careful not to change the sign by cutting off ALL the sign bits.

Data Representation

- In C/C++ variables can be of different types and sizes
 - Integer Types (signed and unsigned)

C Type	Bytes	Bits	ATmega328
[unsigned] char	1	8	byte
[unsigned] short [int]	2	16	word
[unsigned] long [int]	4	32	_1
[unsigned] long long [int]	8	64	_1
int	? ²	? ²	? ²

¹Can emulate but has no single-instruction support

²Varies by compiler/machine (avr-gcc: int = 2 bytes, g++ for x86: int = 4-bytes)

- Floating Point Types

C Type	Bytes	Bits	ATmega328
float	4	32	N/A ¹
double	8	64	N/A ¹

Taking the Negative

- Given a number in signed magnitude or 2's complement how do we find its negative
(i.e. $-1 * X$)
 - Signed Magnitude: Flip the sign bit
 - $0110 = +6 \Rightarrow 1110 = -6$
 - 2's complement: "Take the 2's complement"
 - $0110 = +6 \Rightarrow -6 = 1010$
 - Operation defined as:
 1. Flip/invert/not all the bits (1's complement)
 2. Add 1 and drop any carry (i.e. finish with the same # of bits as we start with)

2's Complement System Facts

- Normal binary place values but MSB has negative weight
- Even though MSB has a weight, it still determines the sign of the number
 - (0 = positive / 1 = negative)
- Special Numbers
 - **0** = All 0's (00...00), **-1** = All 1's (11...11)
 - **Max Positive** = 0 followed by all 1's (011..11)
 - **Max Negative** = 1 followed by all 0's (100...00)
- To take the negative of a number
(e.g. -7 => +7 or +2 => -2), requires taking the complement
 - 2's complement of a # is found by flipping bits and adding 1

1001	$x = -7$
0110	Bit flip (1's comp.)
+ 1	Add 1
0111	$-x = -(-7) = +7$